

```

1 #=====
2 # MIT 6.046J 代码精读 • PS4 - O(1) 摊还最小队列
3 # 源文件: minqueue.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS4 Problem 4-1: Extreme FIFO Queue with O(1) amortized FIND-MIN.
7
8 A FIFO queue of distinct integers supporting ENQUEUE, DEQUEUE, FIND-MIN, each in
9 O(1) amortized time (any m operations run in O(m) total).
10
11 Design (the "monotonic-deque min-queue"):
12 * a real FIFO ``queue`` (deque) holding the elements in arrival order;
13 * an auxiliary ``mono`` deque holding a *non-decreasing* subsequence of the
14   current queue contents -- exactly the elements that could ever become the
15   minimum in the future.
16
17 Invariants:
18 I1. ``mono`` is a subsequence of ``queue`` in the same (arrival) order.
19 I2. ``mono`` is non-decreasing front-to-back.
20 I3. mono[0] == min(queue). (front of mono is always the current minimum.)
21
22 ENQUEUE(x): push x on queue; pop from the *back* of mono every element >= x
23   (they can never be the min while x is present and newer), then push x on mono.
24 DEQUEUE(): pop the front of queue; if it equals mono's front, pop mono's front.
25 FIND-MIN(): return mono's front.
26
27 Amortization: each element is pushed to / popped from ``mono`` at most once over
28 its lifetime, so the total mono-work across m operations is O(m); the queue work
29 is O(1) worst case each. Hence O(1) amortized per operation.
30 """
31 from __future__ import annotations
32
33 from collections import deque
34 from typing import Deque
35
36
37 class MinQueue:
38     def __init__(self) -> None:
39         self._q: Deque[int] = deque()      # arrival order (the true FIFO)
40         self._mono: Deque[int] = deque()   # non-decreasing candidates for the min
41
42     def __len__(self) -> int:
43         return len(self._q)
44
45     def enqueue(self, x: int) -> None:
46         self._q.append(x)
47         while self._mono and self._mono[-1] > x:
48             self._mono.pop()
49         self._mono.append(x)
50
51     def dequeue(self) -> int:
52         if not self._q:
53             raise IndexError("dequeue from empty queue")
54         x = self._q.popleft()
55         if self._mono and self._mono[0] == x:
56             self._mono.popleft()
57         return x
58
59     def find_min(self) -> int:
60         if not self._mono:
61             raise IndexError("find_min of empty queue")
62         return self._mono[0]
63
64     # for testing / introspection
65     def as_list(self):
66         return list(self._q)
67

```